

AgentRAG: Enterprise Knowledge Assistant

Complete Technical Project Architecture & Interview Masterclass Guide

Target Role: AI Software Developer / GenAI Engineer | **Tech Stack:** Python, Gemini 2.5 Flash, LangChain, LangGraph, ChromaDB, MySQL 8.0, FastAPI, Streamlit, Docker

1. The 60-Second Elevator Pitch (How to Introduce the Project)

When an interviewer says 'Tell me about your project' or 'Walk me through what you built', deliver this structured 60-90 second response:

'AgentRAG is an enterprise knowledge assistant that solves the problem of enterprise data fragmentation. In most organizations, information is split across unstructured documents—like PDF policies, guidelines, and handbooks—and structured relational databases—like employee and payroll tables in MySQL. Standard LLMs hallucinate when asked company-specific questions and cannot directly query SQL safely.

I built a decoupled 3-tier system:

- **FastAPI REST API** backend powering query processing and document ingestion.
- **LangGraph Orchestrator** that dynamically analyzes the user's intent and routes between a RAG pipeline and a safe Text-to-SQL engine.
- **RAG Pipeline:** Uses LangChain with Google's 3072-dimensional *gemini-embedding-001* model and a persistent ChromaDB vector store. It enforces strict grounding—if an answer isn't in the documents, it explicitly tells the user rather than hallucinating.
- **Text-to-SQL Engine:** Generates MySQL queries with Gemini, passes them through a custom AST/regex validator enforcing read-only SELECT permissions (blocking any DROP, DELETE, or injection), and executes them on MySQL.
- **Streamlit Frontend:** An interactive UI that lets users upload PDFs, ask questions, inspect the selected route, and view exact document citations with page numbers.
- The entire stack is containerized with **Docker Compose** with persistent volumes and verified by 21 automated pytest unit tests.'

2. Complete End-to-End Project Flow

AgentRAG processes queries through two completely distinct sub-pipelines controlled by a centralized LangGraph state machine. Understanding every step of both pipelines is critical for technical rounds.

A. Route 1: Document RAG Flow (Unstructured PDF Documents)

- **Step 1 (Upload):** User submits a PDF through Streamlit, which issues a multipart/form-data HTTP POST to `/documents/upload` on FastAPI.
- **Step 2 (Extraction):** FastAPI validates that the extension is `.pdf`, saves the file to `data/documents/`, and invokes `PyPDFLoader` to parse text and extract page numbers.

- **Step 3 (Chunking):** RecursiveCharacterTextSplitter breaks pages into chunks of **800 characters** with **120 characters overlap**, keeping paragraphs and list items intact while adding 1-indexed page metadata.
- **Step 4 (Embedding):** Chunks are passed to Google's gemini-embedding-001, converting each text chunk into a 3072-dimensional vector.
- **Step 5 (Indexing):** Vectors, text chunks, and metadata (source filename, page number) are stored in ChromaDB in chroma_data/.
- **Step 6 (Retrieval):** When a question arrives, ChromaDB performs cosine similarity search retrieving the top 4 most relevant chunks.
- **Step 7 (Grounded Synthesis):** Context and query are sent to gemini-2.5-flash at temperature 0.0 with a strict instruction: answer ONLY from context, or return 'I could not find this information in the available documents.'
- **Step 8 (Citation Display):** The response returns the synthesized answer, the route badge ('rag'), and structured sources (filename, page number, text excerpt).

B. Route 2: MySQL Text-to-SQL Flow (Structured Relational Data)

- **Step 1 (Intent Classification):** LangGraph's router_node sends the question to Gemini with a routing prompt. Questions about departments, employee names, counts, salaries, or hire dates classify as sql.
- **Step 2 (Query Generation):** sql_node supplies Gemini with the database schema context (tables, columns, foreign keys, sample rows) to generate a MySQL SELECT query.
- **Step 3 (Security Validation):** The raw SQL is passed to validate_sql_query(). It verifies the query starts strictly with SELECT or WITH, confirms no destructive keywords (DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE) exist, and ensures no multiple statements exist via semicolons.
- **Step 4 (Database Execution):** Validated query executes against MySQL via SQLAlchemy. If no rows match, it returns an explicit friendly message.
- **Step 5 (Natural Language Synthesis):** Raw SQL rows, query, and user question are passed to Gemini to generate a clean conversational summary.
- **Step 6 (UI Display):** Streamlit displays the answer, the 'sql' route badge, and an expandable inspection box showing the exact SELECT query executed.

C. LangGraph State Machine Architecture

LangGraph models the execution workflow as a deterministic Directed Acyclic Graph (DAG). The shared state is defined as a typed dictionary:

```
class AgentState(TypedDict):
    question: str          # User query
    route: str             # 'rag' or 'sql'
    context: str           # Retrieved document excerpts
    sql_query: str         # Generated and validated SELECT statement
    sql_result: str        # Raw rows from MySQL
    answer: str            # Final response text
    sources: List[Dict[str, Any]] # Document citations (filename, page, snippet)
```

3. Deep Dive into Core Concepts & Design Decisions

Interviewers frequently test your fundamental understanding of *why* technologies were selected over alternatives.

A. What is RAG and Why is it Used?

Retrieval-Augmented Generation (RAG) is an architectural pattern that enhances an LLM's output by dynamically retrieving authoritative facts from an external knowledge base before generating a response. Instead of relying solely on the training weights of the LLM, the model uses retrieved context to formulate its answer.

Why RAG is essential in enterprise systems:

1. **Prevents Hallucinations:** By restricting the LLM to retrieved context (grounding), the model is prevented from fabricating facts.
2. **Dynamic Knowledge Updates:** Uploading a new company policy takes 2 seconds (chunk + embed). Fine-tuning an LLM would cost thousands of dollars and hours of compute.
3. **Data Privacy & Access Control:** Proprietary internal documents stay in your local vector database rather than leaking into external model training sets.
4. **Auditability & Source Attribution:** RAG allows citing the exact document name, page number, and section, building trust with business stakeholders.

B. Chunking Strategy: Recursive Character Splitting

A naive character splitter cuts text at arbitrary character counts (e.g. every 500 characters), which slices words and breaks sentences mid-thought. We used RecursiveCharacterTextSplitter with **separators**: `['\\n\\n', '\\n', '•', ':', ',', ' ', '']`.

- **Chunk Size (800 characters):** Large enough to contain a complete corporate policy rule or clause, but small enough to remain semantically focused without diluting vector similarity.
- **Chunk Overlap (120 characters / ~15%):** Ensures context continuity across boundaries so sentences spanning the edge of a chunk are never severed from their meaning.

C. What is ChromaDB, Why Use It, and What Are the Alternatives?

ChromaDB is an open-source, lightweight, AI-native vector database designed for developer velocity and local/embedded persistence.

Why ChromaDB was chosen for AgentRAG:

- **Embedded Architecture:** Runs in-process inside the Python runtime or locally on disk (persist_directory) without requiring an external cluster or cloud subscription.
- **Native Metadata Filtering:** Stores and filters custom metadata (document filename, page number) alongside dense embeddings seamlessly.
- **Python First & Seamless LangChain Integration:** Provides direct support through langchain-chroma.

Comprehensive Vector Database Comparison Table:

Vector DB	Type	Pros	Cons / Trade-offs	Best Used For
ChromaDB (Used)	Embedded / Client-Server	Zero config, local disk persistence, free, open-source	Not designed for billion-scale sharding across clusters	Prototyping, mid-sized enterprise assistants, single-node apps

FAISS (Facebook)	In-memory Vector Index	Extremely fast C++ similarity search, GPU acceleration	No built-in CRUD, no persistence server, manual metadata mapping	Raw high-speed similarity search on static vector datasets
Pinecone	Fully Managed Cloud SaaS	Zero ops, auto-scaling, high availability, sub-50ms latency	Proprietary, closed-source, monthly recurring cost, data leaves premises	Cloud-first enterprise applications requiring turnkey infrastructure
Qdrant / Milvus	Distributed Open-Source DB	Scales to billions of vectors, rich payload filtering, cloud & self-hosted	Heavy operational overhead to manage Kubernetes clusters	Large-scale production enterprise multi-tenant search engines
pgvector (PostgreSQL L)	Relational DB Extension	Combines ACID relational SQL data and vectors in the same DB	Slower index builds (IVFFlat/HNSW) at massive scale vs dedicated vector DBs	Apps already heavily built on PostgreSQL wanting unified storage

D. Why LangGraph Instead of Simple If-Else or LangChain Chains?

In a naive script, routing is often written as `if 'salary' in q: run_sql() else: run_rag()`. While tempting for a quick prototype, this fails on nuanced questions (e.g. *'What is the refund policy for engineering software?'* has 'engineering' but is a policy question).

- **Deterministic State Management:** LangGraph formalizes execution with an explicit TypedDict state, tracking variables systematically across nodes.
- **Decoupled Node Logic:** Each node (Router, RAG, SQL, Formatter) is a pure, testable function that can be unit tested in isolation.
- **Observable & Inspectable:** In an interview, LangGraph demonstrates understanding of stateful AI workflows, conditional branching, and enterprise agent architecture.
- **Extensibility:** Adding human-in-the-loop validation, memory checkpoints, or retry loops requires adding an edge without rewriting procedural spaghetti code.

E. Text-to-SQL Pipeline & Multi-Tier Database Security

Allowing an LLM to query a production database is hazardous without strict defensive guardrails. We implemented a **Zero-Trust Security Validator** (backend/database/sql_validator.py):

1. **Leading Keyword Constraint:** Queries must strictly begin with SELECT or WITH. Any other leading command is rejected immediately.
2. **Destructive Command Blacklist:** Regex inspection blocks INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, REPLACE, GRANT, REVOKE, and EXEC.
3. **Anti-Chaining / Semicolon Rejection:** Blocks query stacking attacks (e.g. `SELECT * FROM employees; DROP TABLE employees;`).
4. **Sanitization:** Strips markdown code blocks, inline comments (`--`, `#`, `/* */`), and whitespace.

4. Comprehensive Interview Q&A; Masterclass (25+ Questions)

Study these exact questions and answers to master your technical interview rounds:

- **Q1: Walk me through the high-level architecture of AgentRAG.**

Answer: AgentRAG uses a decoupled 3-tier architecture: (1) A Streamlit frontend for document uploads and chat, communicating strictly over HTTP; (2) A FastAPI backend handling REST endpoints (/health, /documents/upload, /chat); and (3) A LangGraph state machine orchestrating intent routing between a LangChain + ChromaDB RAG pipeline and a MySQL Text-to-SQL engine with Google Gemini 2.5 Flash as the underlying LLM. The entire stack runs via Docker Compose with persistent storage volumes.

- **Q2: Why did you separate Streamlit and FastAPI instead of writing everything inside Streamlit?**

Answer: Putting business logic inside Streamlit creates tight coupling and severe scalability issues. Streamlit re-executes the entire script on every user interaction, which would cause redundant DB connections and state re-initialization. By isolating core AI workflows in a FastAPI REST API, the backend remains stateless, performant, and reusable—we could easily connect a React dashboard, a mobile app, or a Slack bot to the same API without altering any backend logic.

- **Q3: How does the system handle health checks and service monitoring?**

Answer: The FastAPI backend exposes a GET /health endpoint that pings the MySQL database (SELECT 1) and verifies the ChromaDB vector store directory and document chunk count. If the database is unreachable, it gracefully reports 'status: degraded' and 'database: disconnected' with the specific error, allowing orchestration layers or frontends to respond cleanly.

- **Q4: What is the exact step-by-step lifecycle of a document uploaded to AgentRAG?**

Answer: When a user uploads a PDF: (1) FastAPI validates the file extension and saves the PDF to disk; (2) PyPDFLoader extracts raw page text and page metadata; (3) RecursiveCharacterTextSplitter divides pages into 800-character chunks with 120-character overlap; (4) Gemini's gemini-embedding-001 model converts chunks into 3072-dimensional vector embeddings; and (5) Chunks, vectors, and metadata (filename, 1-indexed page number) are indexed into ChromaDB. The endpoint returns the chunks indexed count.

- **Q5: Why did you choose 800 characters chunk size and 120 characters overlap?**

Answer: Chunk size represents a trade-off between semantic granularity and contextual completeness. In enterprise policy documents, 800 characters (~150-180 words) matches the typical length of a complete corporate clause or policy paragraph. The 120-character overlap (~15%) prevents 'boundary truncation'—ensuring that if a condition or prerequisite spans across two chunks, neither chunk loses the crucial connecting context.

- **Q6: How do you guarantee the RAG assistant never hallucinates on missing information?**

Answer: Through prompt engineering guardrails and low-temperature inference. The RAG prompt explicitly commands Gemini: 'Answer the question relying ONLY on the provided context excerpts. If the answer cannot be determined directly from the context, respond EXACTLY with: "I could not find this information in the available documents." Do not guess or extrapolate.' Temperature is set to 0.0 for deterministic factual responses. Furthermore, if vector retrieval returns 0 documents, this fallback string is returned automatically.

• **Q7: What embedding model did you use, what is its dimensionality, and how does vector search work?**

Answer: We use Google's 'gemini-embedding-001' model, which generates 3072-dimensional dense embeddings. When a user asks a question, the query text is embedded into the same 3072-dimensional mathematical vector space. ChromaDB uses cosine similarity distance (or dot product) to calculate the mathematical angle between the query vector and all stored chunk vectors, retrieving the top 4 chunks with the smallest angular distance.

• **Q8: What is the difference between Dense Retrieval (Embeddings) and Sparse Retrieval (BM25)?**

Answer: Dense retrieval (vector embeddings) understands semantic meaning, synonyms, and paraphrasing (e.g. matching 'annual leave' with 'paid vacation' even if words differ). Sparse retrieval (like BM25 or TF-IDF) matches exact keyword frequencies, which is superior for part numbers, IDs, and domain-specific acronyms. In our project we used dense retrieval via Gemini embeddings, though in production a hybrid search (Dense + BM25 with Reciprocal Rank Fusion) is ideal.

• **Q9: What is the 'Lost in the Middle' problem in RAG, and how do you mitigate it?**

Answer: The 'Lost in the Middle' phenomenon (demonstrated in NLP research by Liu et al.) shows that LLMs are most attentive to information at the very beginning and very end of their input context, frequently overlooking facts buried in the middle of long prompts. We mitigate this by keeping top-k focused (k=4 chunks of 800 chars = ~3200 characters), formatting each chunk with clear bracketed headers '[Excerpt X | Source: ..., Page: ...]', and placing the question immediately after the context.

• **Q10: Explain the LangGraph state machine in this project.**

Answer: LangGraph orchestrates our workflow as a state machine. It begins at START and passes an AgentState dictionary into the 'router' node. The router invokes Gemini to classify the question into 'sql' or 'rag'. A conditional edge checks state['route']: if 'rag', it transitions to 'rag_node' (ChromaDB retrieval + synthesis); if 'sql', it transitions to 'sql_node' (query generation + validation + execution). Both flow into 'finalize_response' before terminating at END.

• **Q11: Why is an LLM-based router better than keyword matching?**

Answer: Keyword matching is brittle and prone to false triggers. For instance, the query 'What is the refund policy for engineering software licenses?' contains the word 'engineering' (a department name) but is conceptually a customer policy question. An LLM-based router understands semantic intent and context, correctly routing it to RAG. However, as a defensive measure, our router also includes a rule-based fallback if the LLM output is ambiguous.

• **Q12: Could this workflow have been built with standard LangChain LCEL chains? Why LangGraph?**

Answer: While simple branching is possible with LangChain Expression Language (LCEL) RunnableBranch, LangGraph offers superior architectural discipline: (1) It enforces an explicit, centralized state schema; (2) Nodes are pure Python functions that are trivial to mock and test independently; and (3) It provides a foundation to add complex patterns like cycles, human validation checkpoints, and multi-step retries without refactoring the code into an unmanageable chain.

• **Q13: How does the Text-to-SQL node know what tables and columns exist in MySQL?**

Answer: The backend provides schema context directly in the system prompt via get_database_schema_context(). This includes exact table definitions (departments, employees), data types, primary and foreign key constraints, and 3-5 representative sample rows. This gives the LLM precise structural grounding to generate correct JOINS on departments.department_id = employees.department_id and format date/salary aggregations accurately.

• Q14: Walk me through your SQL validation and security checks in detail.

Answer: Our `sql_validator.py` module implements multiple defensive layers: (1) Clean query: strips markdown fences (````sql`), comments (`--` and `/* */`), and whitespace; (2) Single statement verification: rejects any semicolon to prevent stacked injection queries; (3) Keyword blacklist: checks against regex patterns for DROP, DELETE, INSERT, UPDATE, ALTER, TRUNCATE, CREATE, REPLACE, GRANT, REVOKE, and EXEC; and (4) Command check: confirms the first word is strictly SELECT or WITH. If any check fails, execution is aborted and a security error is returned.

• Q15: What happens if a user submits: 'How many employees earn more than 100000; DROP TABLE employees;'

Answer: The query enters the router and is routed to the SQL node. Gemini might generate the stacked query or clean it. Regardless of what the LLM generates, before execution the string hits `validate_sql_query()`. The validator detects the embedded semicolon and the blacklisted 'DROP' keyword, rejects the query with 'Security Error: Destructive SQL command forbidden: 'DROP'', and the database is never touched.

• Q16: How do you handle queries where the database returns zero matching records?

Answer: When a query executes cleanly but matches no rows (e.g. searching for a non-existent department or an employee who doesn't exist), the SQL node inspects 'if not results:' and immediately returns: 'No records found matching your query criteria in the database.' This avoids passing an empty list to Gemini, which could otherwise cause the model to hallucinate or speculate.

• Q17: How does Docker Compose ensure MySQL is fully initialized before FastAPI starts?

Answer: MySQL takes 5-10 seconds to start and initialize tables on first boot. If FastAPI starts immediately, its startup healthcheck will fail. We resolved this using Docker Compose 'service_healthy' dependencies. The MySQL service defines a healthcheck executing 'mysqladmin ping'. In the backend service, we specify: 'depends_on: mysql: condition: service_healthy'. This guarantees FastAPI only launches after MySQL is verified ready to accept TCP connections.

• Q18: How is data persisted when containers are stopped or removed?

Answer: We use Docker named volumes and host bind-mounts: (1) 'mysql_data' volume mapped to `/var/lib/mysql` preserves all MySQL table schemas and records; (2) 'chroma_data' volume mapped to `/app/chroma_data` preserves the vector store index and chunk embeddings; and (3) A host bind-mount `./data/documents:/app/data/documents` ensures that uploaded PDF source files persist on the host filesystem.

• Q19: How do you manage API keys and credentials securely in this project?

Answer: All sensitive credentials—including GEMINI_API_KEY, MYSQL_USER, and MYSQL_PASSWORD—are managed exclusively through environment variables and a local `.env` file. We provide a sanitized `.env.example` template for setup. The `.env` file, persistent vector databases (`chroma_data/`), and temporary files are explicitly added to `.gitignore`. A git commit scan confirms that zero secrets or API keys are committed to source control.

• Q20: What did you test using pytest, and how did you mock external services?

Answer: We wrote 21 automated pytest tests across 5 test suites: (1) `test_health.py` tests `/health` for healthy and degraded database states using `unittest.mock.patch`; (2) `test_sql_validator.py` tests valid SELECT queries, destructive query rejections (DROP, DELETE, TRUNCATE), comment stripping, and chained statement injections; (3) `test_router.py` tests intent classification and LangGraph edge routing; (4) `test_rag.py` tests context formatting, source deduplication, grounded answers, and not-found fallbacks; and (5) `test_chat_api.py` tests FastAPI request validation, 400 errors for empty questions and non-PDF uploads, and response schemas.

• Q21: What are the primary architectural limitations of this project?

Answer: Being designed as a clean, interview-ready project, it intentionally omits: (1) User Authentication & RBAC (any user can query all employee salaries); (2) Multi-document cross-filtering (ChromaDB collection is global rather than partitioned per user/department); (3) Distributed vector scaling (ChromaDB is running locally rather than a multi-node Qdrant or Milvus cluster); and (4) Hybrid BM25 keyword search.

• Q22: If you had 2 more weeks to prepare this for production, what would you add first?

Answer: I would prioritize three enhancements: (1) Add OAuth2 / JWT authentication with Role-Based Access Control (RBAC) so that junior staff cannot query executive compensation in MySQL; (2) Implement a Cross-Encoder Re-ranker (e.g. BGE-Reranker or Cohere) to re-score the top 20 retrieved chunks down to the top 4 most relevant excerpts before prompt synthesis; and (3) Add streaming responses via Server-Sent Events (SSE) so users see tokens stream in real time.

• Q23: How would you scale the database component if the enterprise had 500 tables instead of 2?

Answer: Passing 500 table schemas in an LLM prompt would exceed context limits, increase token costs, and confuse the model. In a large enterprise setup, I would implement Dynamic Schema Pruning: (1) Embed table descriptions and column names into a vector database; (2) For a user query, perform a similarity search to retrieve only the top 3-5 relevant table schemas; and (3) Inject only those candidate tables into the Text-to-SQL generation prompt.

• Q24: What embedding dimension does gemini-embedding-001 use and why does dimensionality matter?

Answer: gemini-embedding-001 produces 3072-dimensional vectors. Dimensionality represents the capacity of the vector to encode subtle nuances of language—higher dimensionality allows capturing complex semantic distinctions (e.g. fine legal or technical phrasing), though it requires more memory and compute for distance calculations compared to smaller 384-dim or 768-dim models (like all-MiniLM-L6-v2).

• Q25: What is the difference between RAG and Fine-Tuning, and when would you use each?

Answer: RAG provides external, up-to-date knowledge to an LLM at inference time. Fine-tuning adjusts the internal weights of a model to adapt its style, tone, format, or specialized task behavior (e.g. generating a specific JSON schema or learning medical terminology). Rule of thumb: 'Use Fine-Tuning to teach the model how to act; use RAG to teach the model what to know.'

5. Morning-of-the-Interview Quick Cheat Sheet

Review these key technical parameters and numbers 30 minutes before your interview:

Metric / Parameter	Value / Specification	Why It Matters in Interview
LLM Model	Gemini 2.5 Flash	Low latency, high throughput, zero temperature for grounding
Embedding Model	gemini-embedding-001 (3072 dims)	High semantic capacity for enterprise policy documents
Chunk Size & Overlap	800 chars / 120 chars (~15%)	Prevents broken clauses while maintaining semantic focus
Top-K Retrieval	k = 4 chunks	Avoids 'Lost in the Middle' prompt bloat
Vector Store	ChromaDB (local persistent)	Zero cloud cost, in-process, persistent to chroma_data/
Relational Database	MySQL 8.0 (25 rows, 5 depts)	Relational integrity with foreign keys and realistic test data
SQL Validator	SELECT / WITH only + Blacklist	Prevents SQL injection, DROP, DELETE, and stacked queries
Orchestrator	LangGraph (StateGraph)	Stateful routing DAG with clear conditional edges
Backend API	FastAPI (/health, /upload, /chat)	Async I/O, Pydantic validation, OpenAPI documentation
Frontend UI	Streamlit (Pure HTTP Client)	Decoupled presentation tier with zero business logic
Test Suite	21 tests via pytest (100% pass)	Mocked external APIs, unit tests for security and routing

6. Resume Bullets Ready to Copy-Paste

AgentRAG – Enterprise Knowledge Assistant | *Python, FastAPI, LangChain, LangGraph, Gemini 2.5 Flash, ChromaDB, MySQL, Streamlit, Docker*

- Architected a dual-source enterprise knowledge assistant using **LangGraph** to dynamically route natural language queries between an unstructured document RAG pipeline and a structured MySQL database.
- Implemented a zero-hallucination document RAG pipeline using **LangChain**, **gemini-embedding-001** (3072 dimensions), and **ChromaDB**, achieving grounded generation with exact page citations and a deterministic fallback guardrail.
- Developed a secure Text-to-SQL engine using Gemini 2.5 Flash with custom AST/regex validation, strictly enforcing read-only SELECT permissions and rejecting destructive statements (DROP/DELETE) and SQL injection attacks.
- Decoupled backend business logic into a **FastAPI** REST API and built an interactive **Streamlit** UI, containerizing the complete system (MySQL, backend, frontend) with **Docker Compose** and achieving 100% test coverage across 21 pytest unit tests.